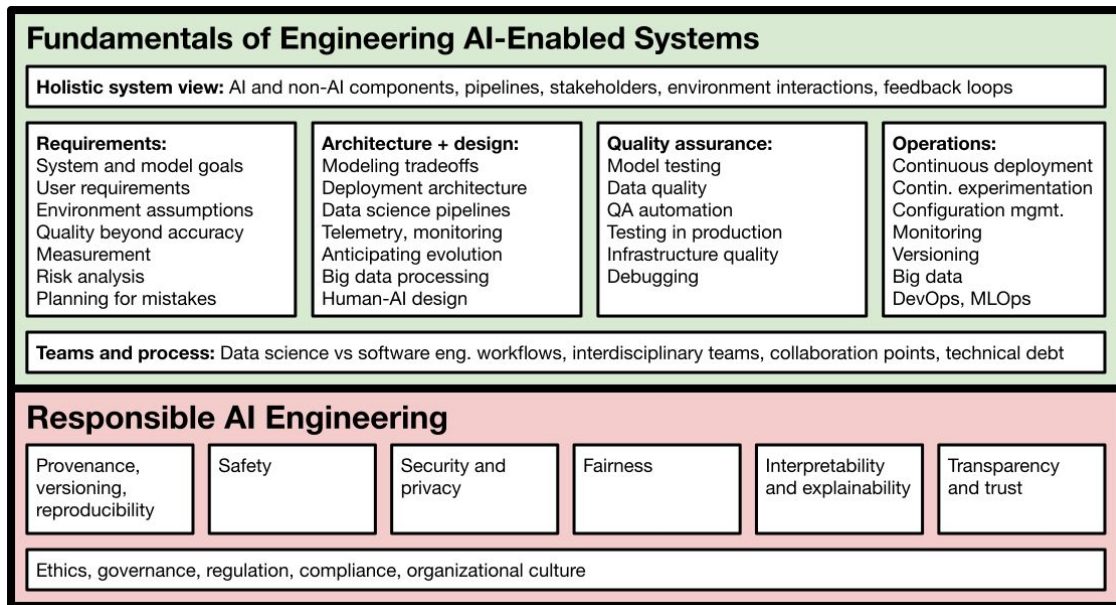




Machine Learning in Production

Versioning and Provenance

Foundational Technology for Responsible Engineering



Learning Goals

- Judge the importance of data provenance, reproducibility and explainability for a given system
- Create documentation for data dependencies and provenance in a given system
- Propose versioning strategies for data and models

Case Study: Credit Scoring



DHH



@dhh · [Follow](#)



The [@AppleCard](#) is such a fucking sexist program. My wife and I filed joint tax returns, live in a community-property state, and have been married for a long time. Yet Apple's black box algorithm thinks I deserve 20x the credit limit she does. No appeals work.

3:34 PM · Nov 7, 2019



22K



Reply



Copy link

[Read 1.1K replies](#)



DHH   · Nov 8, 2019

[@dhh](#) · [Follow](#)

Replying to @dhh

I wasn't even pessimistic to expect this outcome, but here we are: [@AppleCard](#) just gave my wife the VIP bump to match my credit limit, but continued to be an utter fucking failure of a customer service experience. Let me explain...



DHH  

[@dhh](#) · [Follow](#)

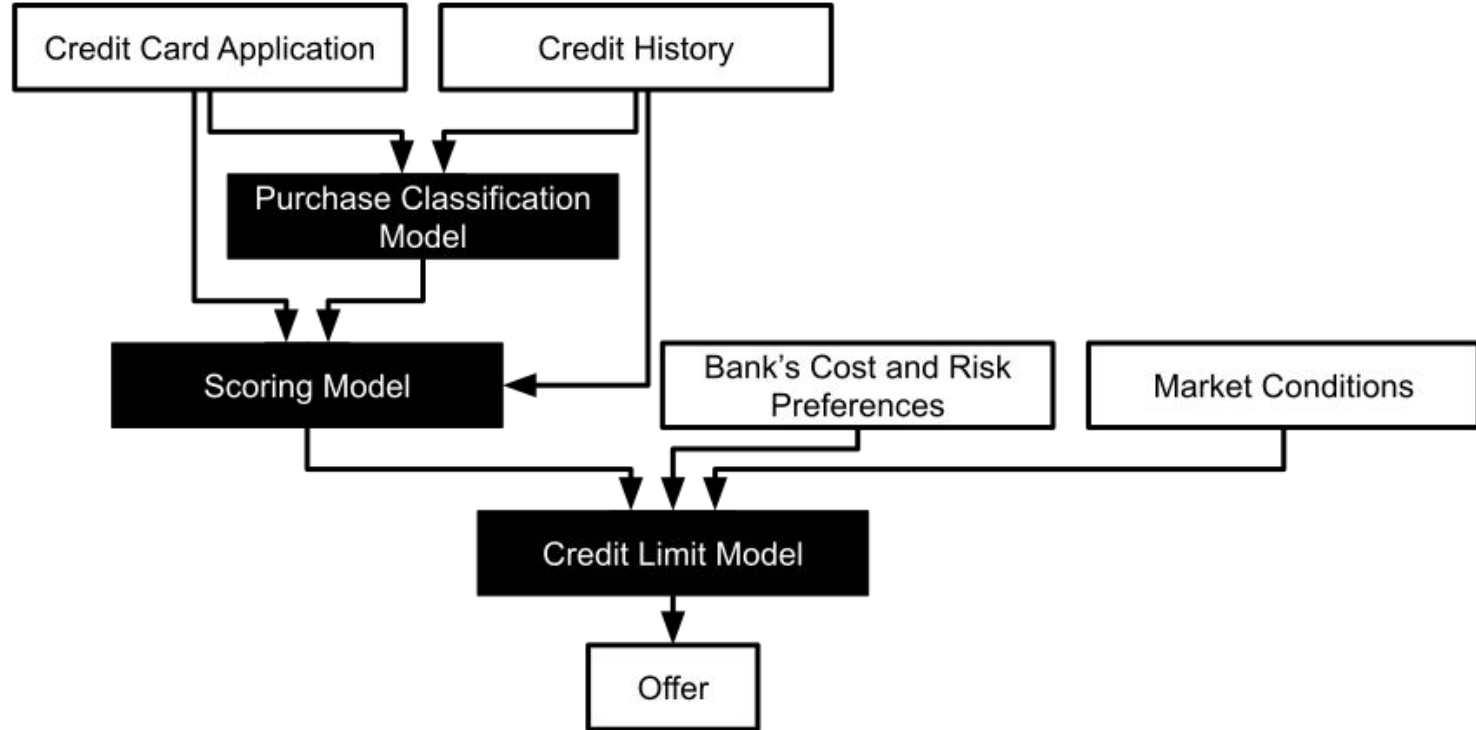
She spoke to two Apple reps. Both very nice, courteous people representing an utterly broken and reprehensible system. The first person was like "I don't know why, but I swear we're not discriminating, IT'S JUST THE ALGORITHM". I shit you not. "IT'S JUST THE ALGORITHM!"

6:20 PM · Nov 8, 2019

 3.5K  Reply  Copy link

[Read 51 replies](#)

- What model was used? Can we reproduce?
- What caused the issue? What data was used?



Debugging?



What went wrong? Where? How to fix?

Debugging Questions beyond Interpretability

- Can we reproduce the problem?
- What were the inputs to the model?
- Which exact model version was used?
- What data was the model trained with?
- What pipeline code was the model trained with?
- Where does the data come from? How was it processed/extracted?
- Were other models involved? Which version? Based on which data?
- What parts of the input are responsible for the (wrong) answer? How can we fix the model?

Practical Data and Model Versioning

- Versioning the data
- Versioning the model
- Trace back the data used to train a model, even after we make some changes to the data

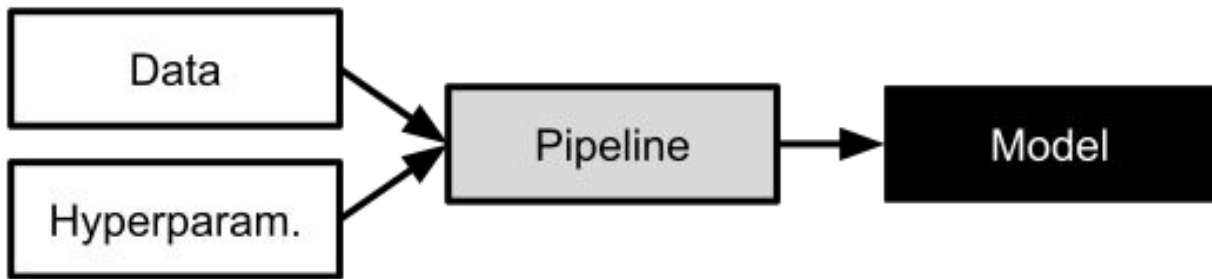
How to version large datasets?

```
CREATE TABLE customer (  
  customer_id      UUID PRIMARY KEY DEFAULT,  
  first_name       VARCHAR(100) NOT NULL,  
  last_name        VARCHAR(100) NOT NULL,  
  date_of_birth    DATE NOT NULL,  
  ssn_hash         VARCHAR(64),  
  government_id    VARCHAR(50),  
  email            VARCHAR(255),  
  phone            VARCHAR(20),  
  address          VARCHAR(200),  
  account_opened_date DATE DEFAULT CURRENT_DATE,  
  customer_segment VARCHAR(50)  
  ...  
);
```

Versioning Strategies for Datasets

1. Store copies of entire datasets (like Git), identify by checksum
2. Store deltas between datasets (like SVN, Mercurial)
3. History of individual database records (e.g. S3 bucket versions)
 - a. some databases specifically track provenance (who has changed what entry when and how)
 - b. specialized data science tools eg [Hangar](#) for tensor data
4. Offsets in append-only database (like Kafka), identify by offset
5. Version pipeline to recreate derived datasets ("views", different formats)
 - a. e.g. version data before or after cleaning?

Versioning Models: Multiple Parts!



Associate model version with:

- pipeline code version
- data version
- hyperparameters

Traditional models: blobs

Larger models: formats like SafeTensor (can version deltas).

API models: version configuration

ML Versioning Tools (MLOps)

Tracking **data**, **pipeline**, and **model versions**

Modeling pipelines: inputs and outputs and their versions

- automatically tracks how data is used and transformed

Often tracking also metadata about versions

- Accuracy
- Training time
- ...

Versioning Dependencies

Pipelines depend on many frameworks and libraries

Ensure reproducible builds

- Declare versioned dependencies from stable repository (e.g. requirements.txt + pip)
- Avoid floating versions
- Optionally: commit all dependencies to repository ("vendoring")

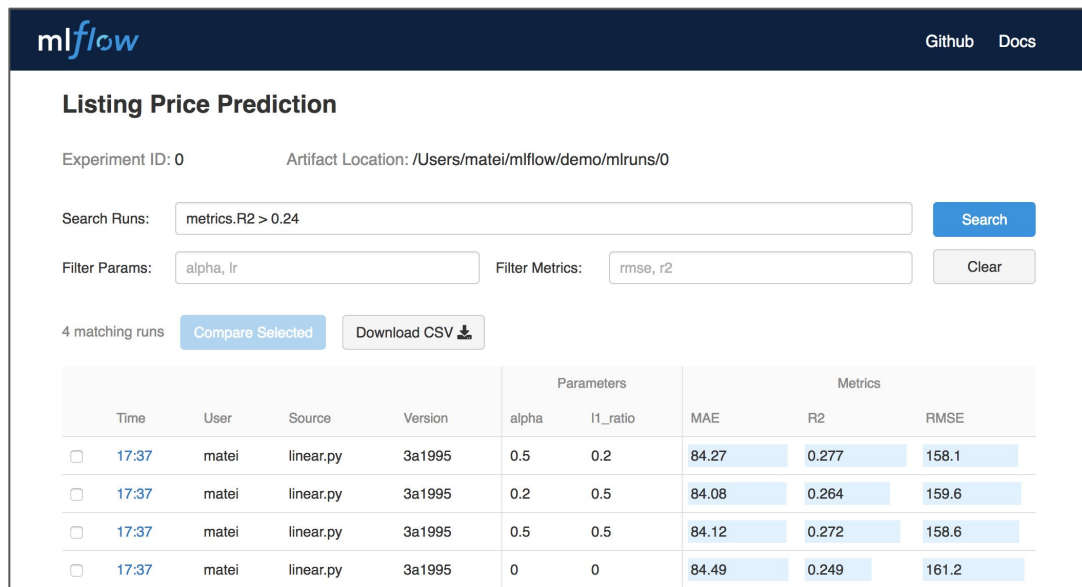
Optionally: Version entire environment (e.g. Docker container)

Test build/pipeline on independent machine (container, CI server, ...)

Example w/ Metadata: Experiment Tracking

Log information within pipelines:
hyperparameters used,
evaluation results, and model files

Many tools: MLflow, Neptune, TensorBoard, Weights & Biases ...



Example: DVC (Data Version Control)

- Tracks models and datasets, built on Git
- Define a sequence of steps (e.g. data processing, training, evaluation) in a reproducible pipeline, incrementalization
- Orchestrates learning in cloud resources: Detect changes in any step and re-run only the necessary parts when there's an update

```
dvc add images
dvc run -d images -o model.p cnn.py
dvc remote add myrepo s3://mybucket
dvc push
```

DVC Example

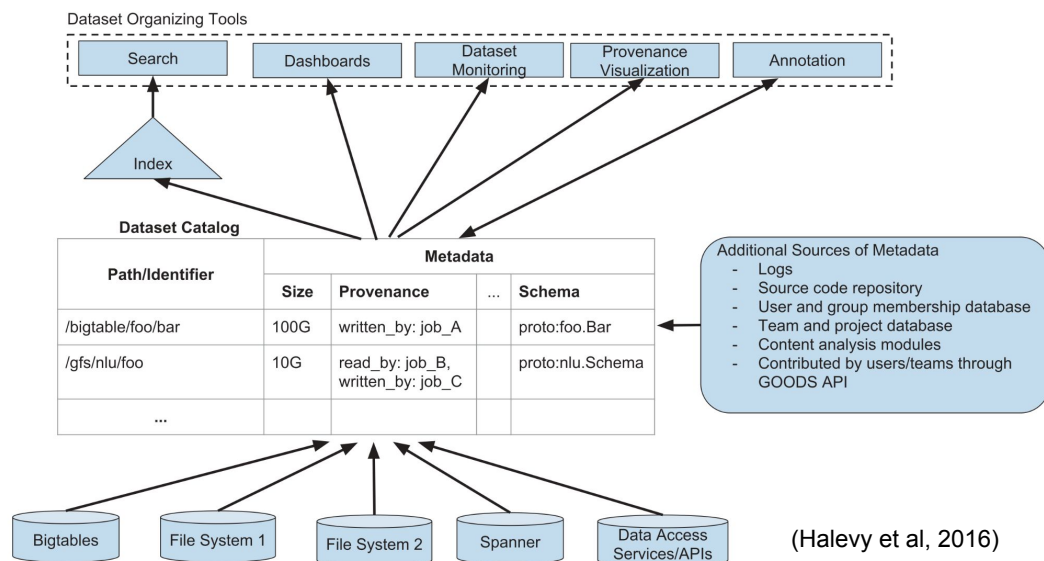
- Higher complexity and constraints
- More automation

```
stages:
  features:
    cmd: jupyter nbconvert --execute featurize.ipynb
    deps:
      - data/clean
    params:
      - levels.no
    outs:
      - features
    metrics:
      - performance.json
  training:
    desc: Train model with Python
    cmd:
      - pip install -r requirements.txt
      - python train.py --out ${model_file}
    deps:
      - requirements.txt
      - train.py
      - features
    outs:
      - ${model_file}:
          desc: My model description
    plots:
      - logs.csv:
          x: epoch
          x_label: Epoch
    meta: 'For deployment'
# User metadata and comments are supported
```

Google's Goods

Automatically derive data dependencies from system logs

Track metadata for each table



No manual tracking/dependency declarations needed

Requires homogeneous infrastructure

Similar systems for tracking inside databases, MapReduce, Sparks, etc.

From Model Versioning to Deployment

Decide which model version to run where

- automated deployment and rollback (cf. canary releases)
- Kubernetes, Cortex, BentoML, ...

Track which prediction has been performed with which model version (logging)

Logging and Audit Traces

Key goal: If a customer complains about an interaction, can we reproduce the prediction with the right model? Can we debug the model's pipeline and data? Can we reproduce the model?

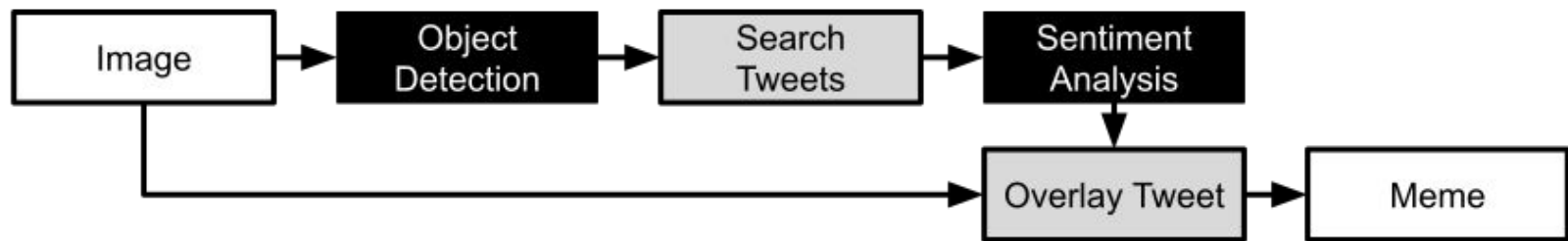
- Version everything
- Record every model evaluation with model version
- Append only, backed up

`<date>,<model>,<model version>,<feature inputs>,<output>`

`<date>,<model>,<model version>,<feature inputs>,<output>`

`<date>,<model>,<model version>,<feature inputs>,<output>`

Logging for Composed Models



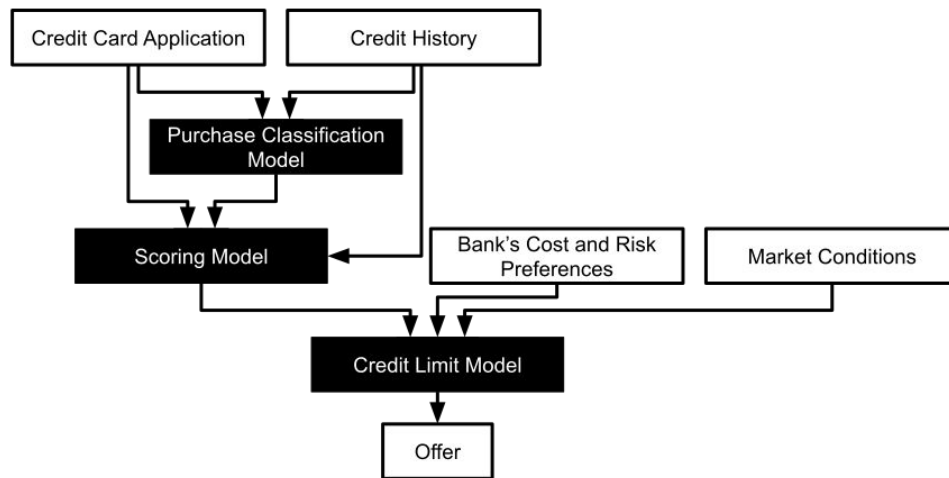
Ensure all predictions are logged

Provenance Tracking

Historical record of data and its origin

Data Provenance

- Track origin of all data
 - Collected where?
 - Modified by whom, when, why?
 - Extracted from what other data or model or algorithm?
- ML models often based on data derived from many sources through many steps, including other models



Versioning vs Provenance

Feature	Data Versioning	Data Provenance
Purpose	Tracks changes in data over time	Documents data's origin, lifecycle, usage
Primary Focus	Data states / changes between versions	Data lineage, sources, and transformations
Granularity	Focuses on entire datasets or files	Tracks individual data entries and transformations
Application	Rollbacks, collaborative editing	Compliance, auditing, trust, and traceability
Example	Version 1.0, 1.1, 2.0 of a dataset	Source, timestamp, consent for each entry

Tracking Data Lineage

Document all data sources

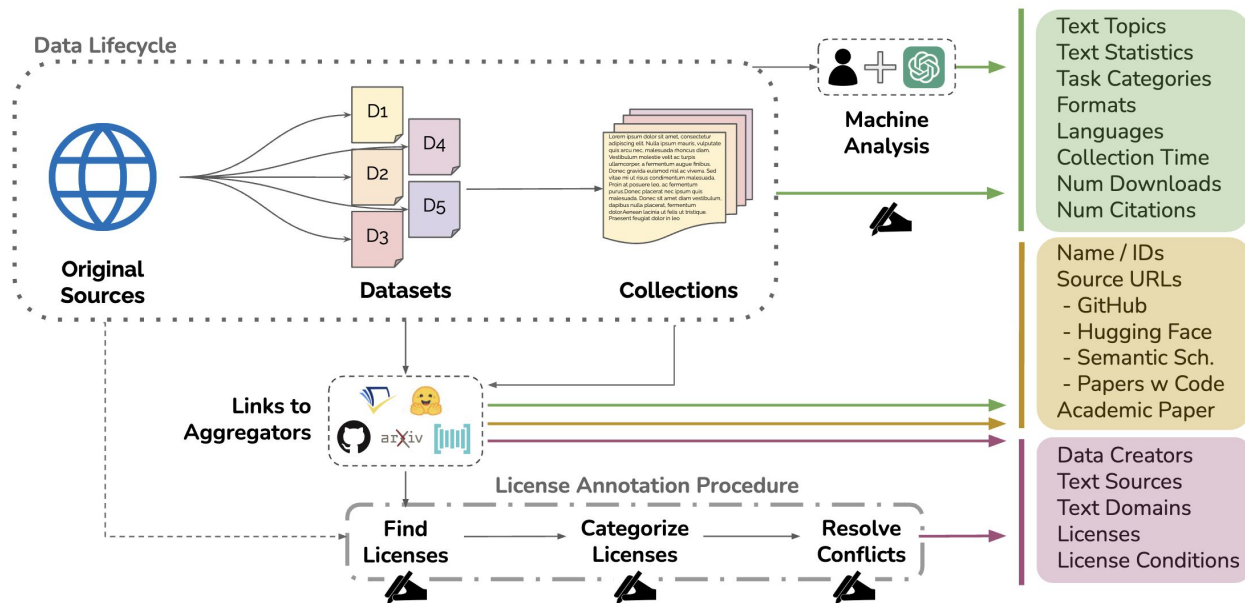
Identify all model dependencies and flows

Ideally model all data and processing code

Avoid "visibility debt"

(Advanced: Use infrastructure to automatically capture/infer dependencies and flows as in [Goods](#))

Example: Data Provenance Initiative



Key idea: Need to recover data provenance for existing datasets

Example: Data Provenance Initiative

Observe issues in data provenance:
Many datasets are annotated to have
more permissive licenses than they
actually have!

CORRECT LICENSE		LICENSE ACCORDING TO AGGREGATORS (AGG.)				
LICENSE	COUNT	AGG.	COMM.	UNSPEC.	NON-COMM.	ACAD.-ONLY
Commercial	856 (46.1%)	🔒	349	507	0	0
		👉	176	677	1	2
		🔗	313	520	1	22
Unspecified	570 (30.7%)	🔒	112	458	0	0
		👉	164	395	6	5
		🔗	31	523	1	15
Non-Commercial	352 (19.0%)	🔒	49	303	0	0
		👉	113	152	80	7
		🔗	2	191	157	2
Academic-Only	80 (4.3%)	🔒	9	71	0	0
		👉	9	65	2	4
		🔗	5	65	2	8
Total	1858 (100%)	🔒	519 (28%)	1339 (72%)	0 (0%)	0 (0%)
		👉	462 (25%)	1289 (69%)	89 (5%)	18 (1%)
		🔗	351 (19%)	1299 (70%)	161 (9%)	47 (3%)

Feature Provenance

How are features extracted from raw data?

- during training
- during inference

Has feature extraction changed since the model was trained?

Recommendation: Modularize and version feature extraction code

Example?

Feature Store

Stores feature extraction code as functions, versioned

Catalog features to encourage reuse

Compute and cache features centrally

Use same feature used in training and inference code

Advanced: Immutable features -- never change existing features, just add new ones (e.g., creditscore, creditscore2, creditscore3)

Model Provenance

How was the model trained?

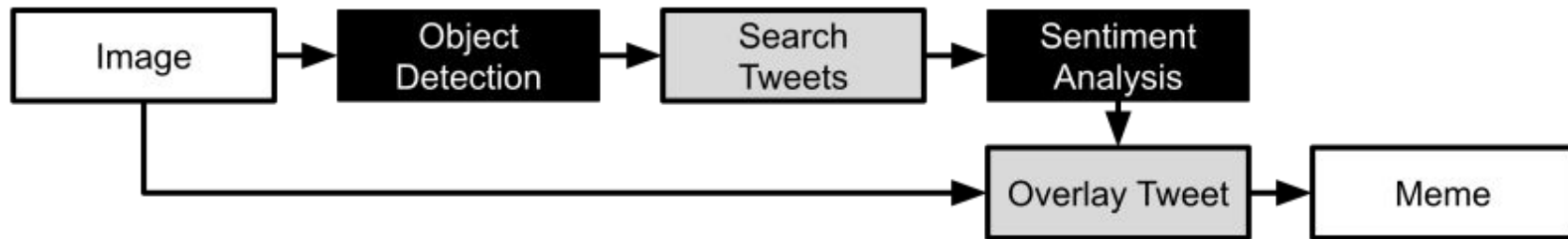
What data? What library? What hyperparameter? What code?

Ensemble of multiple models?

Mixem: provenance in practice

Feature	Model Card	Unified Model Record (UMR)
Purpose	Summarizes model's purpose, limitations, and usage	Tracks full model lifecycle and operations
Content Focus	Usage guidelines, ethical considerations, limitations	Technical specs, lifecycle history, compliance records
Compliance	Limited focus on regulatory compliance	Aligned with governance and regulatory standards
Maintenance	Updated occasionally, typically per major update	Continuous updates with lifecycle events and audits

In Real Systems: Tracking Provenance Across Multiple Models



Version all models involved!

Provenance for Foundation Models?

LLaVA-1.6 Vicuna 13B

Unified Model Record

Released: 2023-03-30

MODEL DETAILS

General information about the model

Science QA is a dataset designed for evaluating and training models in the task of answering science-related questions.

PACKAGES

- llama2
- mm-instruct-data
- llama2-13b
- gpt4
- LLaVA-1.6-Vicuna-13B
- science-qa
- coco
- vicuna-13b
- CC3M
- openai-gpt
- vicuna-70k-gpt4
- LLaVA
- vicuna

Model Card

INTENDED USE

- Research and academic purposes to advance the field of natural language processing and chatbot design.
- Developers and enthusiasts to explore chatbot technologies and implement in non-commercial projects.
- Educational institutions for teaching concepts related to AI and chatbots.
- Non-profit organizations to enhance their customer support or engagement through chatbots.
- Open-source community contribution and improvement.

FACTORS

- Performance comparison with other models like ChatGPT and Google Bard.
- Cost-effectiveness of training the model.
- Accessibility of the model for non-commercial use.
- Potential for the community to contribute to model improvement.
- Ease of integration into existing systems for developers.

EVALUATION DATA

- 70k user-shared ChatGPT conversations were used for fine-tuning.

Page 1 of 1

• GPT-4 used for preliminary evaluation to benchmark against ChatGPT and Google Bard.

• Responses compared for detailed and well-structured answers.

• Eight question categories devised to assess various aspects of chatbot performance.

• Comparison based on helpfulness, relevance, accuracy, and detail of responses.

TRAINING DATA

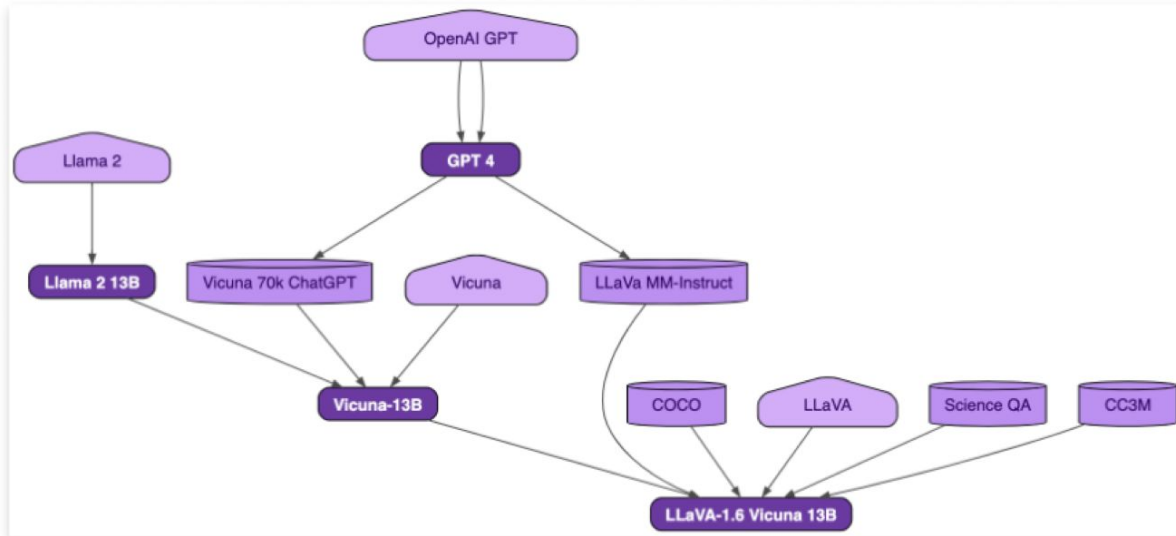
- Approximately 70K user-shared conversations gathered from ShareGPT.com.
- HTML content converted back to markdown to filter and maintain data quality.
- Lengthy conversations divided into smaller segments to fit model's maximum context length.
- Dataset expanded to ensure Vicuna understands long context.
- Innovations such as gradient checkpointing and flash attention applied to manage GPU memory requirements efficiently.

ETHICAL CONSIDERATIONS

- Ensure the model does not generate harmful or biased content by incorporating adequate safety measures.
- Be transparent about the model's capabilities and limitations to users and stakeholders.
- Consider the environmental impact of training large models and strive for more energy-efficient methodologies.
- Respect user privacy and data protection laws, especially when dealing with conversation data.
- Regularly review and update the model and its usage policies to address evolving ethical standards and societal expectations.

RECOMMENDATIONS

- Use Vicuna-13B for experimental and research purposes to explore limitations and potential improvements.
- Consider the ethical implications of deploying chatbots and adhere to fair use policies.
- Because Vicuna is open-source, contributions such as bug fixes, feature enhancements, and training on diverse datasets are recommended.
- Engage with the development community through discussions, sharing use cases, and providing feedback.
- Stay informed about future updates and versions by following the project's official communication channels.



Wang, K., Iranzad, A. N., Schaffter, S., Risdal, M., Precup, D., & Lebensold, J. (2024). Mitigating Downstream Model Risks via Model Provenance. *arXiv preprint arXiv:2410.02230*.

Breakout Discussion: Movie Predictions

Assume you are receiving complains that a child gets many recommendations about R-rated movies

In a group, discuss how you could address this in your own system and post to #1ecture, tagging team members:

- How could you identify the problematic recommendation(s)?
- How could you identify the model that caused the prediction?
- How could you identify the training code and data that learned the model?
- How could you identify what training data or infrastructure code "caused" the recommendations?

Summary

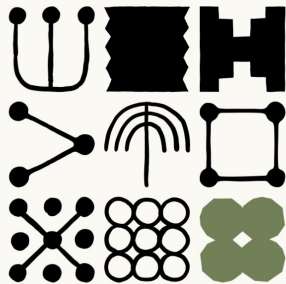
Provenance is important for debugging and accountability

Data provenance, feature provenance, model provenance

Version everything!

- Strategies for data versioning at scale
- Version the entire pipeline and dependencies
- Adopt a pipeline view, modularize, automate
- Containers and MLOps, many tools

Engineering at Anthropic



A postmortem of three recent issues

Published Sep 17, 2025

This is a technical report on three bugs that intermittently degraded responses from Claude. Below we explain what happened, why it took time to fix, and what we're changing.

Between August and early September, three infrastructure bugs intermittently degraded Claude's response quality. We've now resolved these issues and want to explain what happened.

In early August, a number of users began reporting degraded responses from Claude. These initial reports were difficult to distinguish from normal variation in user feedback. By late August, the increasing frequency and persistence of these reports prompted us to open an investigation that led us to uncover three separate infrastructure bugs.

Why detection was difficult

Our validation process ordinarily relies on benchmarks alongside safety evaluations and performance metrics. Engineering teams perform spot checks and deploy to small "canary" groups first.

These issues exposed critical gaps that we should have identified earlier. The evaluations we ran simply didn't capture the degradation users were reporting, in part because Claude often recovers well from isolated mistakes. Our own privacy practices also created challenges in investigating reports. Our internal privacy and security controls limit how and when engineers can access user interactions with Claude, in particular when those interactions are not reported to us as feedback. This protects user privacy but prevents engineers from examining the problematic interactions needed to identify or reproduce bugs.

Each bug produced different symptoms on different platforms at different rates. This created a confusing mix of reports that didn't point to any single cause. It looked like random, inconsistent degradation.

More fundamentally, we relied too heavily on noisy evaluations. Although we were aware of an increase in reports online, we lacked a clear way to connect these to each of our recent changes. When negative reports spiked on August 29, we didn't immediately make the connection to an otherwise standard load balancing change.

What we're changing

As we continue to improve our infrastructure, we're also improving the way we evaluate and prevent bugs like those discussed above across all platforms where we serve Claude. Here's what we're changing:

- **More sensitive evaluations:** To help discover the root cause of any given issue, we've developed evaluations that can more reliably differentiate between working and broken implementations. We'll keep improving these evaluations to keep a closer eye on model quality.
- **Quality evaluations in more places:** Although we run regular evaluations on our systems, we will run them continuously on true production systems to catch issues such as the context window load balancing error.
- **Faster debugging tooling:** We'll develop infrastructure and tooling to better debug community-sourced feedback without sacrificing user privacy. Additionally, some bespoke tools developed here will be used to reduce the remediation time in future similar incidents, if those should occur.

Further Readings

- Sugimura, Peter, and Florian Hartl. "Building a Reproducible Machine Learning Pipeline." *arXiv preprint arXiv:1810.04570* (2018).
- Halevy, Alon, Flip Korn, Natalya F. Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. "Goods: Organizing google's datasets." In Proceedings of the 2016 International Conference on Management of Data, pp. 795-806. 2016.
- Vartak, Manasi, and Samuel Madden. "Modeldb: Opportunities and challenges in managing machine learning models." *IEEE Data Eng. Bull.* 41, no. 4 (2018): 16-25.